BYTE-LEVEL PREDICTION FROM TOKEN LOGITS: REMOVING TOKENIZATION BIAS AT INFERENCE TIME

Anonymous authors
Paper under double-blind review

ABSTRACT

Language models (LMs) operate on tokens produced by a deterministic tokenizer, yet users interact with them through bytes and characters. We study *tokenization bias*: the mismatch between the byte-level distribution implied by a tokenized LM and the token-conditioned distribution that the model is queried with, which is most severe when a prompt ends in the middle of a token. We show how the token-level distribution of an arbitrary tokenized LM can be converted, without retraining, into exact next-byte probabilities by marginalizing over the token sequences that cover the byte prefix. The resulting decoding wrapper is inference-only, and because all models are mapped into a shared byte space, their predictions can be averaged directly. On a synthetic Markov-chain study, the correction recovers the ground-truth transition probabilities. On the HumanEval Random Span infilling benchmark with CodeLlama-7b, byte-level prediction raises the Suffix-Prefix-Middle (SPM) greedy pass rate from 45.0 to 63.9 and matches or slightly exceeds the Prefix-Suffix-Middle (PSM) setting (65.7 vs. 64.9). Byte-level ensembles of two models improve over their best member on several knowledge and code benchmarks, but not on all of them, and can underperform when members differ substantially in capability. Our evidence comes from single-run evaluations without reported variance, so the magnitude of small differences should be interpreted with caution.

1 INTRODUCTION

Modern language models consume sequences of tokens rather than bytes or characters. A deterministic tokenizer, such as byte-pair encoding (BPE) (Sennrich et al., 2016) or a SentencePiece-style segmenter (Kudo & Richardson, 2018), maps a text string to one canonical token sequence, and the model is trained to predict the next token conditioned on that sequence. This design keeps sequences short and computation tractable, but it introduces a mismatch between what the model has learned and what users ask it to do. Users and downstream programs think in characters, bytes, and whole words, and they frequently present prompts whose final characters do not correspond to a token boundary.

This mismatch matters most in code infilling. In Fill-In-The-Middle (FIM) prompting (Bavarian et al., 2022), the prefix and suffix of a source file are placed around a sentinel, and the model generates the missing span. In the Suffix-Prefix-Middle (SPM) template, the prompt ends with the beginning of the prefix text, which may be part of a longer token that the tokenizer would have merged differently had the full word been present. Conditioning on the token sequence obtained by greedily encoding that prefix can then assign zero probability to the correct continuation. We refer to this failure as *tokenization bias*. Figure 4 illustrates the phenomenon on a first-order Markov chain: when the context token is "A", the tokenized model never emits "A" even though the underlying process does so with probability $1 - \alpha$.

A related consequence is that models with different vocabularies cannot be ensembled at the level of their token distributions, because their output spaces are disjoint. A common byte-level space would remove this obstacle.

We address both problems with a single inference-time construction. Our contributions are:

- A formal definition of tokenization bias and a lemma that relates the token-level distribution of a tokenized LM to an exact byte-level distribution, expressed through the set of valid cover encodings of a byte prefix (Section 3).
- An inference-only decoding procedure that searches backwards from the end of the prompt for cover tokens, so that the marginalization does not require enumerating the entire context history. No weights are updated.
- An empirical study covering a synthetic bias test, the HumanEval Random Span FIM benchmark in PSM and SPM modes, comparisons with token healing and token alignment, a beam-width ablation, and byte-space ensembles of general and code models with distinct vocabularies (Section 4).

The results support the central claim that the correction removes the SPM degradation on FIM and enables cross-tokenizer averaging. They also show that the gains on general benchmarks are small and that ensembles are not uniformly beneficial.

2 Related Work

Subword tokenization. Subword vocabularies were popularized for neural machine translation by Sennrich et al. (2016), who applied BPE merges to open-vocabulary translation, and by Kudo (2018), who showed that sampling among multiple segmentations improves robustness. SentencePiece (Kudo & Richardson, 2018) made language-independent subword training practical. Mielke et al. (2021) survey the trade-offs between words, subwords, characters, and bytes. Our work does not change the tokenizer; it asks what a fixed tokenizer's model implies at the byte level.

Tokenizer-free and byte-level models. Byte-level or character-level models avoid the mismatch by construction. ByT5 (Xue et al., 2022) and CANINE (Clark et al., 2022) train encoders or encoder-decoders directly on bytes or characters. These approaches require pretraining new models and incur longer sequences. In contrast, our method starts from pretrained tokenized models and changes only the decoding procedure.

Infilling and prompt-boundary fixes. FIM training (Bavarian et al., 2022) teaches code models to complete the middle span given prefix and suffix, and the HumanEval Random Span benchmark (Chen et al., 2021; Bavarian et al., 2022) evaluates it. Practical fixes for prompts that end inside a token include token healing, which backs up the prompt by a token and constrains the next generation to be consistent with the removed text, and token alignment, which matches characters across subword boundaries. We use these as baselines in our FIM experiments. Our approach differs in that it computes a full next-byte distribution rather than constraining a single token choice.

Ensembling models with different vocabularies. Averaging or voting over models is straightforward when they share a vocabulary. Methods such as LLM-Blender (Jiang et al., 2023b) and knowledge fusion of LLMs (Wan et al., 2024) combine outputs of heterogeneous models, typically by ranking, generative fusion, or distribution-level alignment that requires training. Our ensemble uses only inference-time byte probabilities as a common space.

Models evaluated. We use Llama 2 (Touvron et al., 2023), Mistral-7B (Jiang et al., 2023a), Yi (Young et al., 2024), and Code Llama (Rozière et al., 2023) as tokenized base models. Evaluation uses MMLU (Hendrycks et al., 2021), PIQA (Bisk et al., 2020), Natural Questions (Kwiatkowski et al., 2019), TriviaQA (Joshi et al., 2017), GSM8K (Cobbe et al., 2021), HumanEval (Chen et al., 2021), and MBPP (Austin et al., 2021). GPT-2 (Radford et al., 2019) is the reference architecture for our synthetic transformer.

3 Methodology

3.1 Setup and notation

Let $\mathcal{B}$ denote the set of byte strings and let $\mathcal{V}$ be the vocabulary of a tokenizer with a deterministic detokenization map $\mathrm{detok} : \mathcal{V}^* \to \mathcal{B}$. A tokenized LM defines a distribution $P_{\mathrm{LM}}(t_{m+1} \mid t_{1:m})$

over the next token. Given a byte prefix $\boldsymbol{x} = x_{1:n}$, the token sequences that are consistent with it are those whose detokenization begins with $\boldsymbol{x}$. We write $\mathcal{C}(\boldsymbol{x})$ for the set of such token sequences. Among them, a sequence is a *valid* encoding if the deterministic tokenizer could have produced it from its own byte string, that is, if the merge rules of BPE would not combine adjacent tokens differently. Valid sequences that cover $\boldsymbol{x}$ and whose last token may extend past the end of $\boldsymbol{x}$ are called *cover encodings*.

3.2 Byte-token representation lemma

Under the autoregressive token model, the probability of a byte prefix is the total probability of the token sequences whose byte string begins with that prefix. Restricting this sum to valid encodings removes the probability mass that the tokenizer could never produce for the given text, which is the source of tokenization bias.

> **Lemma (byte-token representation, informal).** For a byte prefix $\boldsymbol{x} = x_{1:n}$ and a deterministic tokenizer, let $\mathcal{E}(\boldsymbol{x})$ be the set of valid cover encodings of $\boldsymbol{x}$. Then the next-byte distribution of the tokenized model is
>
> $$P(x_{n+1} \mid \boldsymbol{x}) \;\propto \sum_{\boldsymbol{t} \in \mathcal{E}(\boldsymbol{x}\, x_{n+1})} P_{\mathrm{LM}}(\boldsymbol{t}), \tag{1}$$
>
> where $P_{\mathrm{LM}}(\boldsymbol{t}) = \prod_i P_{\mathrm{LM}}(t_i \mid t_{1:i-1})$ and the normalization runs over the 256 possible byte values. Equivalently, the conditional probability of the byte is obtained by summing the probabilities of all next tokens whose byte string begins with x_{n+1}, conditioned on each valid encoding of the history.

The lemma follows from partitioning the event "the byte string begins with $\boldsymbol{x}\, x_{n+1}$" by the valid token sequence that realizes it: each realizing sequence contributes its model probability once, and sequences that are not valid for the tokenizer are excluded because they correspond to no text. We do not repeat the full proof here; the construction is determined by the definitions above.

3.3 Cover-encoding search

Enumerating $\mathcal{E}(\boldsymbol{x})$ naively requires considering all segmentations of the entire context, which grows with prompt length. We instead search backwards from the end of the prompt. Starting at the last byte, the search collects tokens of the vocabulary whose byte strings end at the current boundary and are consistent with the remaining prefix; invalid candidates, which the tokenizer's merge rules would never produce for that text, are discarded and the search terminates once every branch has covered the prefix. Figure 3 shows an example with BPE, in which valid and invalid encodings are marked and the algorithm enumerates the cover tokens for a prompt ending mid-word.

3.4 Decoding wrapper and ensembling

At each generation step the wrapper computes the next-byte distribution for every byte value, samples or selects a byte, and appends it to the byte prefix. Multi-byte tokens are completed by repeating the procedure. Because the distribution is over bytes, the wrapper accepts any tokenized model, and no weights are changed. For ensembling, each member model's next-byte distribution is computed in the same byte space and the distributions are averaged with equal weight before sampling. Beam search is supported by tracking byte-level hypotheses and their log-probabilities.

4 Experiments

4.1 Experimental setup

Synthetic chains. We generate data from a first-order and a third-order Markov chain and train a 6-layer GPT-2-style transformer (Radford et al., 2019) on the samples. We compare the conditional distribution of the tokenized model with the byte-level correction against the ground-truth transition probabilities.

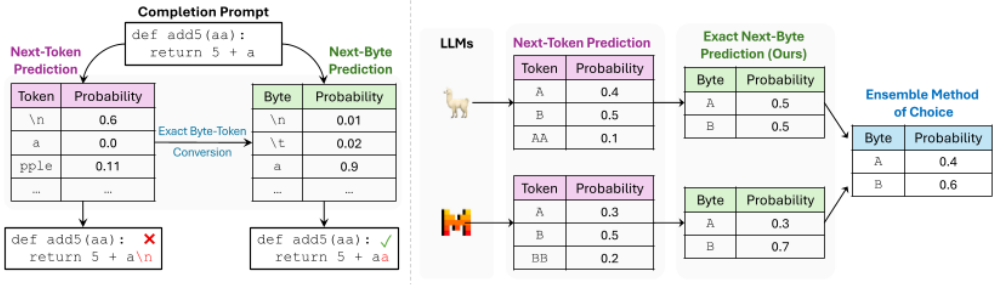


Figure 1: Left: a tokenized LM can suffer tokenization bias when a prompt ends mid-token, as in this code completion example, where the correct completion has zero probability. Our method avoids this problem and predicts the correct token using the same model. Right: our method maps the next-token predictions of arbitrary tokenized LMs to statistically equivalent next-byte predictions, which enables ensemble strategies such as averaging.

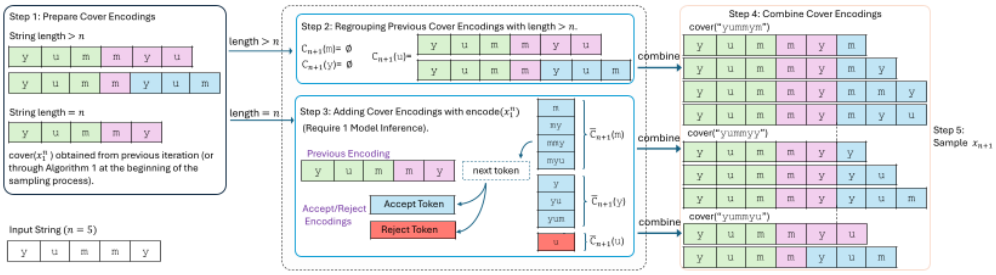


Figure 2: Illustration of the sampling process for x_{n+1} from a tokenized LM $P(t_{k+1} \mid t_{1:k})$. We omit $P(\boldsymbol{t})$ for simplicity. See Section 3.

Code infilling. We use the HumanEval Random Span benchmark (Chen et al., 2021; Bavarian et al., 2022) for infilling Python with CodeLlama-7b. Both PSM and SPM prompt templates are evaluated. Baselines are standard token-level decoding, token healing, and token alignment, the last two applied in SPM mode as specified in our logs. We report greedy pass rate and pass@10 (@10).

General and code benchmarks. For knowledge and reasoning we report accuracy on MMLU, PIQA, Natural Questions, TriviaQA, and GSM8K, comparing token-level and byte-level probabilities for Llama2-7B, Yi-1.5-6B, and Mistral-7B-v0.3. For code generation we report pass@1 on HumanEval and MBPP for CodeLlama2-7b, CodeLlama2-13b, and Yi-Coder-1.5B. The ensemble in each table averages the byte-level distributions of the two best-performing models; "Voting (top-2)" is a majority-vote baseline over the same pair.

Ablation. We vary the beam width over $\{1, 2, 4\}$ for the byte-level FIM model.

Reporting caveats. The logs report one value per configuration. They do not include random seeds, repeated runs, or confidence intervals. For HumanEval (164 problems), one problem corresponds to roughly 0.6 percentage points, so differences of a few tenths of a point should not be interpreted as meaningful.

4.2 Synthetic Tokenization Bias

On the first-order chain, standard tokenization assigns probability 0.0 to the byte A following context A, while the ground truth is $1 - \alpha$ (Figure 4, left). On the third-order chain, the tokenized model's predictive distribution deviates strongly from the ground truth (Figure 4, right). Applying the byte-

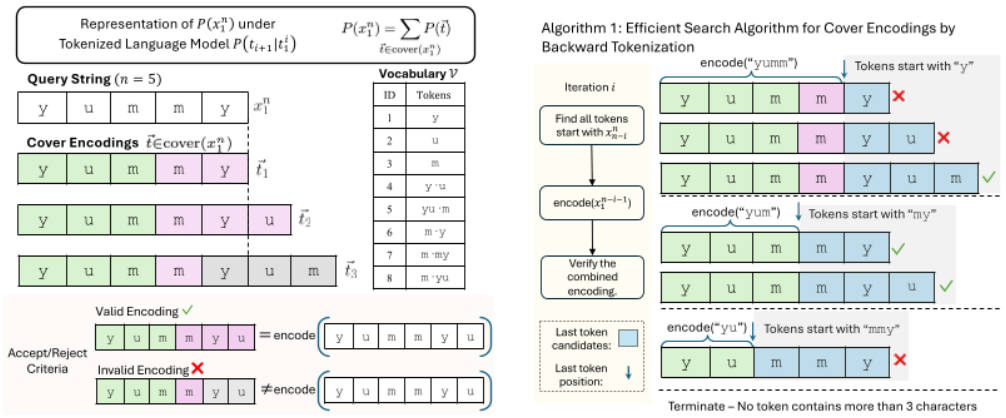


Figure 3: Left: representation of $P(x_{n+1} \mid \boldsymbol{x})$ with a tokenized LM, showing an example of cover encodings and valid and invalid encodings. Right: illustration of the cover-encoding search. Green ticks and red crosses denote valid and invalid encodings. The termination step is omitted from the algorithm for brevity. BPE is used in this example.

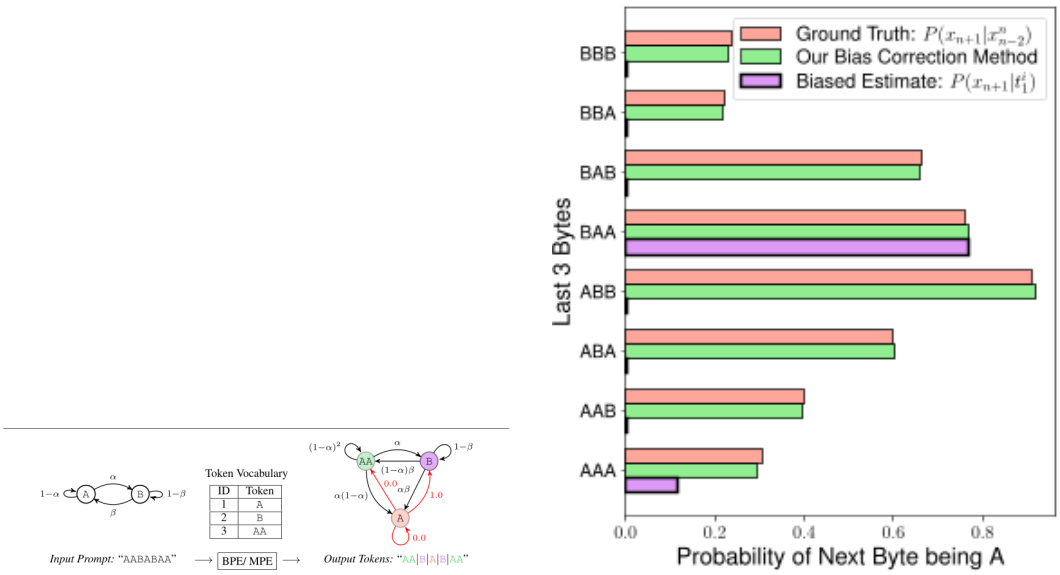


Figure 4: Left: tokenization bias on a first-order Markov chain. Given the context token "A", the tokenized model never samples "A" as the next token, whereas the chain does so with probability $1 - \alpha$. Right: tokenization bias on a third-order Markov chain. The byte-token correction accurately recovers $P(x_{n+1} \mid x_{1:n})$ of the original chain.

token correction recovers $P(x_{n+1} \mid x_{1:n})$ of the original chain in both cases. These experiments confirm that the bias is a property of the decoding interface rather than of the learned chain.

4.3 Code Infilling

Table 1 reports pass rates on HumanEval Random Span. Figure 5 shows the same comparison as a chart. With token-level decoding, the greedy pass rate falls from 64.9 in PSM to 45.0 in SPM, consistent with prompts that end mid-token being out of distribution for the tokenizer. Byte-level prediction reduces this gap: its SPM greedy score is 63.9, close to its PSM score of 65.7. In SPM mode, byte-level prediction also exceeds token healing (62.7) and token alignment (63.0) on greedy decoding, and it obtains the highest @10 score (84.3). In PSM mode, where tokenization issues are

Table 1: Pass rates (%) on the HumanEval Random Span benchmark with CodeLlama-7b. Healing and alignment are heuristic prompt-boundary fixes that apply to SPM only. Best value per column in bold.

Method	PSM Greedy	PSM @10	SPM Greedy	SPM @10
Token-level	64.9	84.0	45.0	66.5
Token-level + Healing	N/A	N/A	62.7	83.8
Token-level + Alignment	N/A	N/A	63.0	82.9
Byte-level (ours)	**65.7**	**84.1**	**63.9**	**84.3**

	PSM		SPM	
Predictions	Greedy	@10	Greedy	@10
Token-level	64.9	84.0	45.0	66.5
+ healing	|	|	62.7	83.8
+ alignment	|	|	63.0	82.9
Byte-level	**65.7**	**84.1**	**63.9**	**84.3**

@10 here is the max over the temperature sweep on the right

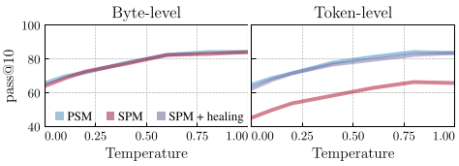


Figure 5: Pass rates for the HumanEval Random Span benchmark with CodeLlama-7b for infilling Python. Across all settings, the byte-level predictions yield better completions. We evaluate the PSM and SPM infilling templates and use token healing and token alignment as baselines. The SPM template forces the model to predict from the middle of a token, which the byte-level predictions handle.

less frequent, byte-level greedy decoding is still slightly higher than token-level decoding (65.7 vs. 64.9), while the @10 scores are nearly identical (84.1 vs. 84.0). Relative to the token-level SPM baseline, the byte-level method improves greedy pass rate by 18.9 points and @10 by 17.8 points.

Qualitatively, the failure cases involve identifiers and keywords such as “delimeter”, “collatz”, and “palindrome” whose prompts end mid-word. Token-level models frequently fail to complete these words, whereas the byte-level method identifies the completion. We did not quantify these cases beyond the qualitative analysis in the logs.

Table 2 shows the beam-width ablation. Increasing the beam from 1 to 4 improves the byte-level model by 2.0 points in PSM (65.7 to 67.7) and 2.9 points in SPM (63.9 to 66.8). The gains are monotone in beam width for both templates, but they come from single runs.

4.4 General and code benchmarks

Table 3 reports the knowledge and reasoning results. Byte-level and token-level scores are nearly identical for most model–task pairs; the largest differences are small (for example, Llama2-7B on GSM8K drops from 15.1 to 12.5, and Mistral-7B on NQ rises from 28.5 to 28.8). The byte-level top-2 ensemble scores 65.4 on MMLU, 80.7 on PIQA, 30.0 on NQ, and 64.2 on TriviaQA, which are higher than the best single model on each of these tasks in the table. On GSM8K the ensemble reaches 55.8, which is below the best member (Yi-1.5-6B, 61.5). The GSM8K result matches the divergence limitation in the logs: Yi-1.5-6B outperforms Mistral-7B-v0.3 by about 20 points on this task, and averaging with a much weaker model dilutes the stronger signal.

Table 4 reports code generation. The byte-level models are not uniformly better: on HumanEval, byte-level pass@1 is lower than token-level for CodeLlama2-7b (28.7 vs. 32.3), CodeLlama2-13b (33.5 vs. 35.8), and Yi-Coder-1.5B (36.5 vs. 38.4). On MBPP, byte-level scores are slightly higher for CodeLlama2-7b (42.4 vs. 40.6) and Yi-Coder-1.5B (53.6 vs. 52.8) and marginally lower for CodeLlama2-13b (47.4 vs. 47.6). The top-2 byte-level ensemble of Yi-Coder-1.5B and CodeLlama2-13b reaches 42.1 on HumanEval, which is 3.7 points (about 9.6% relative) above the best individual model, and 53.6 on MBPP, equal to the best individual score. We take these results to mean that the byte-space conversion does not by itself improve standalone code generation, and that the ensemble benefit is real on HumanEval in this single-run evaluation but not established beyond it.

Table 2: Ablation on beam width for the byte-level model. Metric is HumanEval pass rate (%).

Beam width	PSM	SPM
1	65.7	63.9
2	66.8	65.9
4	67.7	66.8

Table 3: Knowledge and reasoning accuracy (%) with token-level and byte-level probabilities. The top-2 ensemble averages byte-level distributions of the two best models on each task. "N/A" marks settings not reported in the logs. Best byte-level value per column in bold among non-voting rows where the comparison is meaningful.

	MMLU		PIQA		NQ		TriviaQA		GSM8K	
Model	Token	Byte	Token	Byte	Token	Byte	Token	Byte	Token	Byte
Llama2-7B	45.4	45.7	78.1	78.2	25.0	23.4	58.4	58.1	15.1	12.5
Yi-1.5-6B	63.4	63.4	78.5	78.5	22.8	22.7	53.7	53.4	61.3	**61.5**
Mistral-7B-v0.3	62.1	62.1	80.1	80.3	28.5	28.8	63.6	63.5	39.0	39.0
Voting (top-2)	N/A	62.1	N/A	80.1	N/A	N/A	N/A	N/A	N/A	N/A
Top-2 ensemble (ours)	N/A	**65.4**	N/A	**80.7**	N/A	**30.0**	N/A	**64.2**	N/A	55.8

4.5 Analysis and limitations

The results support three conclusions. First, the byte-token correction removes most of the SPM degradation on FIM, and it does so without the heuristics used by the baselines. Second, the correction is essentially free on PSM prompts, where it gives small gains that are within the noise of a single run. Third, byte-space ensembling can help when member models are of comparable strength, as on HumanEval, MMLU, PIQA, NQ, and TriviaQA, but it can hurt when members are far apart, as on GSM8K. These observations are consistent with averaging being a poor combination rule when one member is much weaker.

Several limitations apply. All numbers come from single runs without seeds, repeated evaluation, or confidence intervals, so only the larger effects (for example, the SPM improvement and the HumanEval ensemble gain) should be considered robust. The FIM evaluation covers one model and one benchmark. The lemma is stated at the level of the decoding procedure, and the synthetic experiments are the only setting in which ground-truth byte probabilities are known. The logs do not report wall-clock cost of the backward search, so we cannot quantify the overhead relative to token-level decoding. Finally, the ensemble mixes models whose vocabularies differ, and we did not evaluate weighting schemes beyond uniform averaging.

5 Conclusion

We identified tokenization bias, the mismatch between the byte-level distribution of a tokenized LM and the token-conditioned distribution it is queried with, and showed how to convert token logits into exact next-byte probabilities at inference time by marginalizing over valid cover encodings. The method recovers the ground-truth transitions of synthetic Markov chains, substantially improves SPM infilling on HumanEval Random Span with CodeLlama-7b, and allows byte-space ensembling of models with different vocabularies. Its benefits on standalone code and general benchmarks are small or mixed, and ensembles can underperform when members differ in capability. Future work should repeat the evaluations with multiple seeds and confidence intervals, extend FIM experiments to more models and benchmarks, measure the computational overhead of the backward search, and study learned or capability-aware ensemble weights.

Table 4: Code generation pass@1 (%) on HumanEval and MBPP with token-level and byte-level probabilities. The ensemble uses the top-2 models in byte space.

Model	HumanEval (Token)	HumanEval (Byte)	MBPP (Token)	MBPP (Byte)
CodeLlama2-7b	32.3	28.7	40.6	42.4
CodeLlama2-13b	35.8	33.5	47.6	47.4
Yi-Coder-1.5B	38.4	36.5	52.8	**53.6**
Top-2 ensemble (ours)	N/A	**42.1**	N/A	**53.6**

REFERENCES

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Mohammad Bavarian, Heewoo Jun, Nikolas Tezak, John Schulman, Christine McLeavey, Jerry Tworek, and Mark Chen. Efficient training of language models to fill in the middle. *arXiv preprint arXiv:2207.14255*, 2022.

Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. PIQA: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Jonathan H. Clark, Dan Garrette, Iulia Turc, and John Wieting. CANINE: Pre-training an efficient tokenization-free encoder for language representation. *Transactions of the Association for Computational Linguistics*, 10:73–91, 2022.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023a.

Dongfu Jiang, Xiang Ren, and Bill Yuchen Lin. LLM-Blender: Ensembling large language models with pairwise ranking and generative fusion. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023b.

Mandar Joshi, Eunsol Choi, Daniel Weld, and Luke Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 1601–1611, 2017.

Taku Kudo. Subword regularization: Improving neural network translation models with multiple subword candidates. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 66–75, 2018.

Taku Kudo and John Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations*, pp. 66–71, 2018.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:452–466, 2019.

Sabrina J. Mielke, Zaid Alyafeai, Elizabeth Salesky, Colin Raffel, Manan Dey, Matthias Gallé, Arun Raff, Alexander M. Rush, and Teven Le Scao. Between words and characters: A brief history of open-vocabulary modeling and tokenization in NLP. *arXiv preprint arXiv:2112.10508*, 2021.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.

Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 1715–1725, 2016.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

Fanqi Wan, Xinting Huang, Deng Cai, Xiaojun Quan, Wei Bi, and Shuming Shi. Knowledge fusion of large language models. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.

Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 10:291–306, 2022.

Alex Young, Bei Chen, Chao Li, Chengen Huang, Ge Zhang, et al. Yi: Open foundation models by 01.ai. *arXiv preprint arXiv:2403.04652*, 2024.